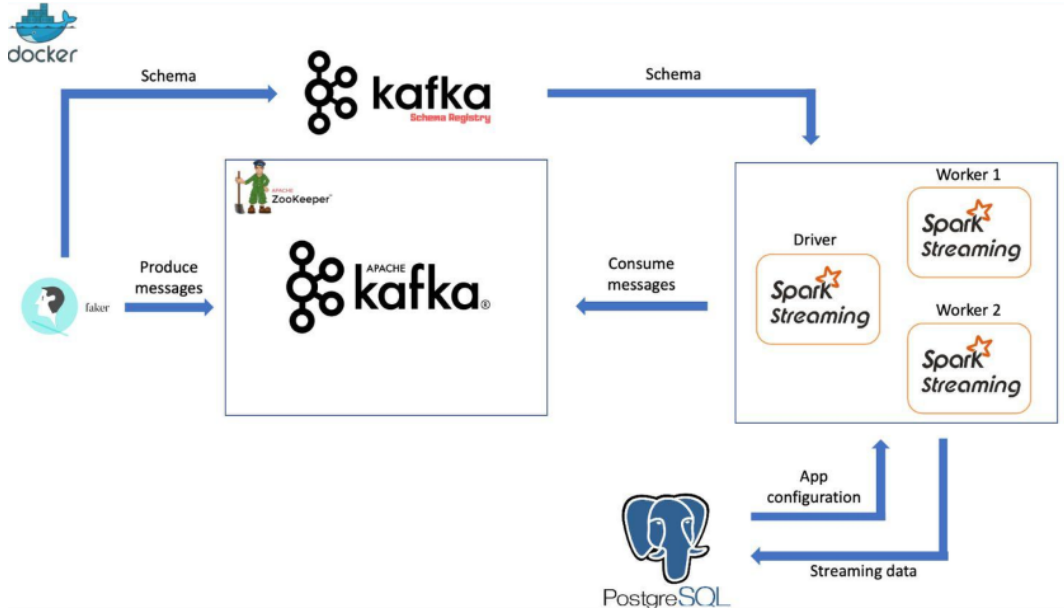
Streaming distribué

Apache Kafka et Spark Structured Streaming

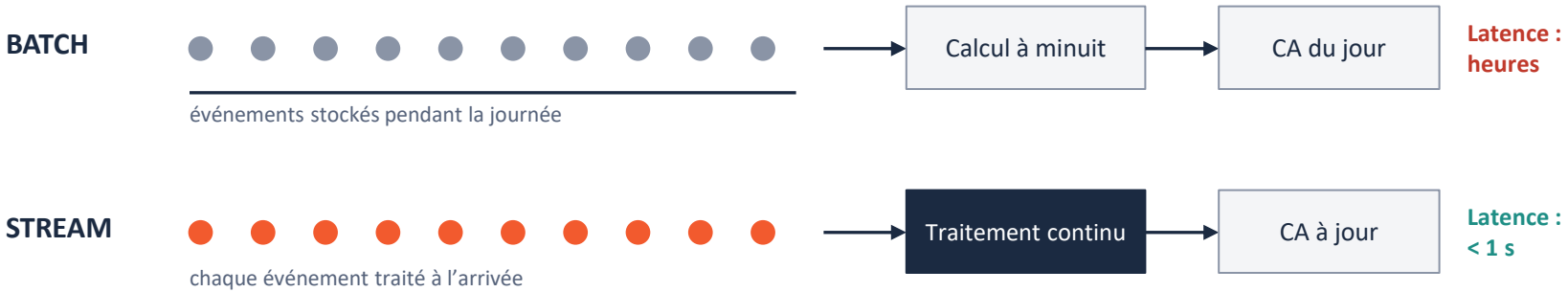
Master 2 Data · 02/11/2026

FORMATEUR : DIALLO ALIMOU



Batch Processing vs Stream Processing

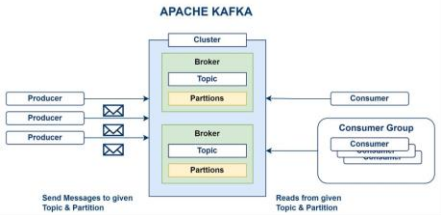
La vraie question : combien de temps entre l'événement et la décision ?



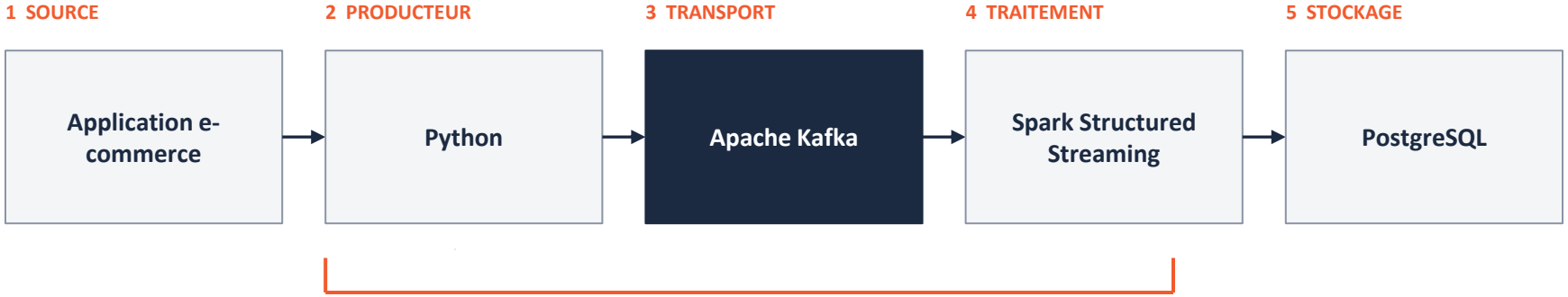
	Batch	Streaming
Exemple	Chiffre d'affaires de la journée calculé à minuit	Chiffre d'affaires mis à jour dès la réception d'un événement de paiement
Latence	Minutes à heures	Millisecondes à quelques secondes

QUESTION
Pourquoi une banque ou une plateforme e-commerce a-t-elle besoin de traiter les événements rapidement ?

Architecture d’un pipeline de streaming



Cinq composants découplés : Python n’a pas besoin de connaître le fonctionnement interne de Spark.



Découplage
Chaque composant évolue, tombe ou est remplacé indépendamment.

Tampon
Kafka absorbe les pics et conserve les événements.

Rejeu
Spark peut relire les événements après une panne.

Introduction à Apache Kafka

Une plateforme distribuée de gestion et de diffusion d'événements.

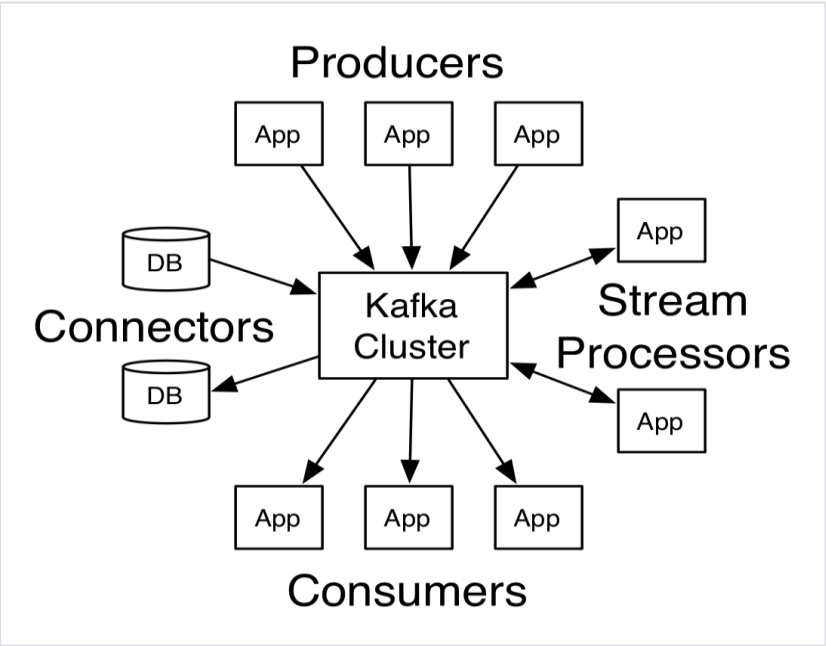


Figure — documentation Apache Kafka

Pourquoi entre producteurs et consommateurs ?
Les deux côtés ne se connaissent pas et n'ont pas le même rythme.

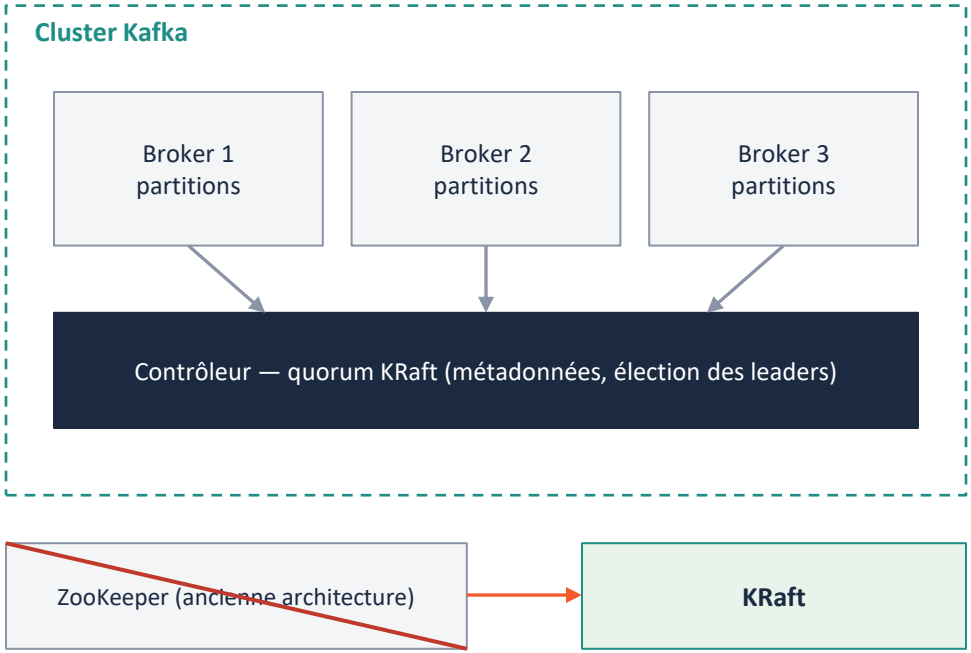
Conserve les événements
Durée de rétention configurable : on peut relire (rejeu).

Plusieurs consommateurs
Chaque groupe lit à son rythme, sans se gêner.

Kafka ≠ moteur de calcul
Spark ou Flink transforment ; Kafka transporte.

Architecture interne de Kafka

Des brokers stockent les partitions ; un contrôleur coordonne le cluster, sans ZooKeeper depuis Kafka 4.0.



Broker
Serveur Kafka qui stocke les partitions.

Cluster
Ensemble de brokers.

Controller
Métadonnées et élections de leaders.

KRaft
Coordination intégrée à Kafka.

ZooKeeper
Ancienne architecture, à connaître historiquement.

Topics, partitions et offsets

Une partition est un journal ordonné ; l'ordre n'est garanti qu'à l'intérieur d'une partition.

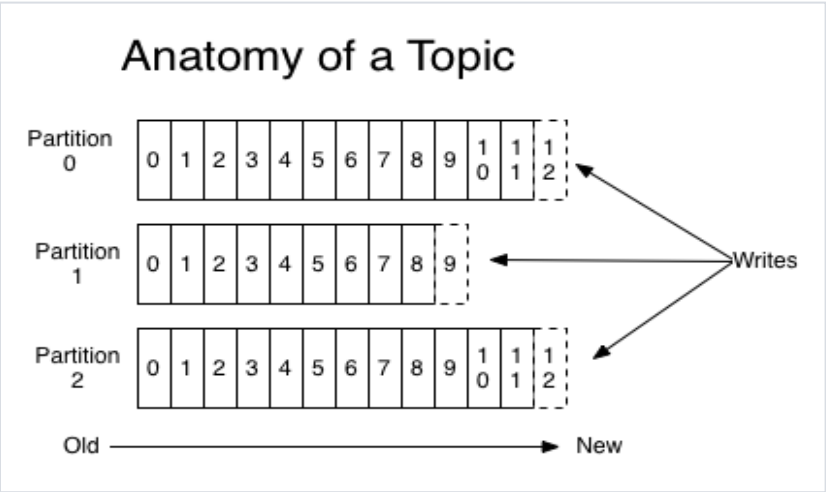


Figure — documentation Apache Kafka — anatomie d'un topic

Topic retail-orders · 3 partitions

Partition	Événements	Offset
P0	ORD001, ORD004	0, 1
P1	ORD002, ORD005	0, 1
P2	ORD003, ORD006	0, 1

ATTENTION
L'ordre est garanti dans une partition, pas globalement entre toutes les partitions.

Topic
catégorie d'événements

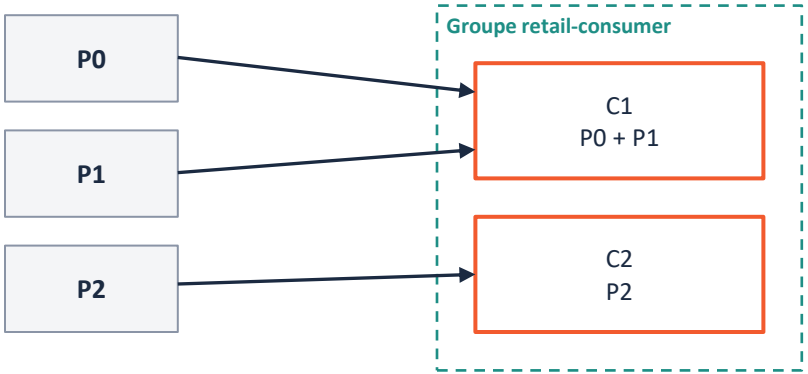
Partition
journal ordonné

Offset
position dans la partition

Ordre
garanti par partition

Producer, Consumer et Consumer Groups

Dans un groupe, chaque partition est lue par un seul consommateur.



Si C2 quitte le groupe : C1 reprend P2 (rebalancing)

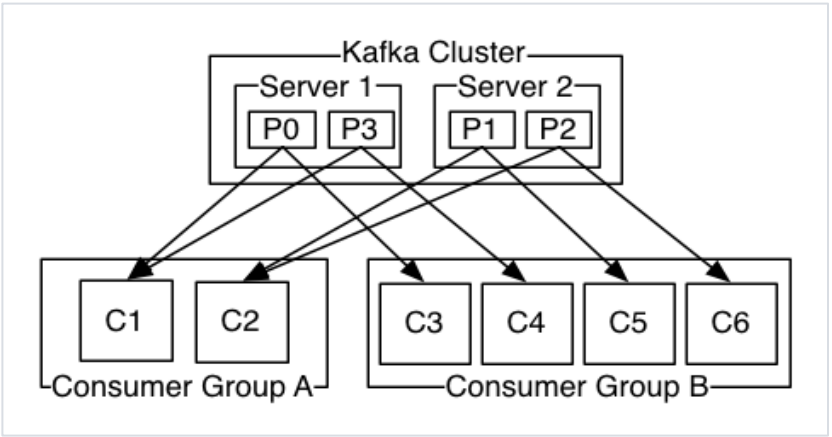


Figure — documentation Apache Kafka — deux groupes, quatre partitions

Producer
publie les événements

Consumer
lit les événements

Consumer Group
consommateurs qui se partagent la lecture

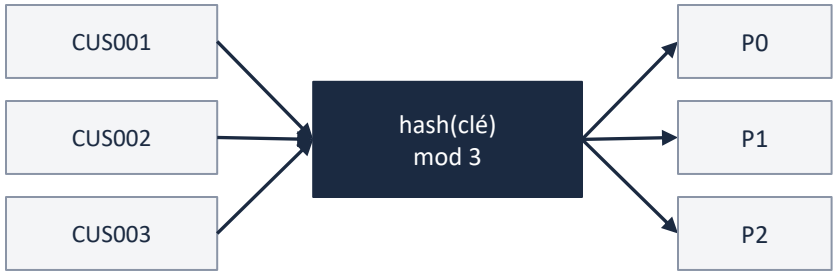
Rebalancing
réattribution des partitions

Les clés de partitionnement

Même clé, même partition : les événements d'un client restent regroupés et ordonnés.

```
{
  "order_id": "ORD001",
  "customer_id": "CUS001",
  "event_type": "ORDER_CREATED"
}
```

clé = customer_id



Clé déséquilibrée : hot partition



Clé équilibrée

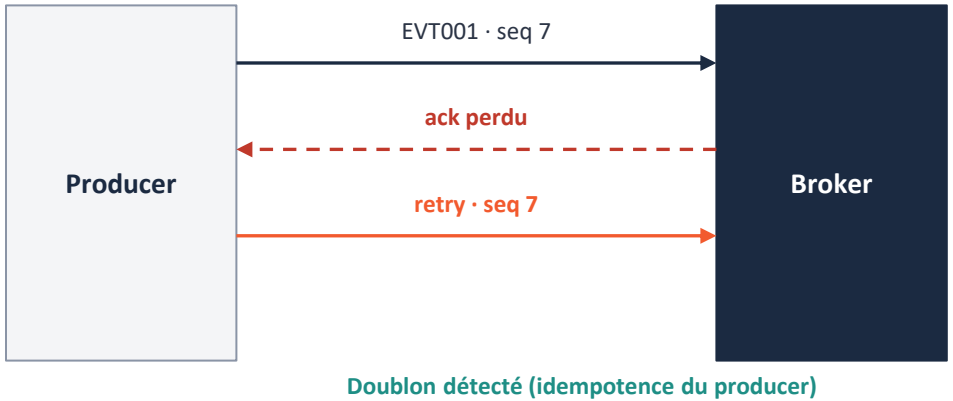


HOT PARTITION
Un client très actif sature une seule partition : un consommateur surchargé pendant que les autres attendent.

Fiabilité et livraison des événements

Une garantie Kafka ne garantit pas l'absence de doublons dans PostgreSQL.

Garantie	Configuration typique	Risque
At-most-once	Pas de nouvelle tentative ; acquittement ou offset validé avant traitement	Perte possible d'événements
At-least-once	Retry + acquittement après traitement	Doublons possibles
Exactly-once	Producteur idempotent + transactions Kafka pour les traitements compatibles	Garantie dans le périmètre Kafka



OFFSET COMMIT
Valider après traitement : at-least-once.

RETRY
Réessayer après un échec : source de doublons.

ATTENTION
Même avec Kafka fiable, PostgreSQL peut recevoir un événement deux fois.

Installation et administration avec Docker Compose

Un broker en conteneur, deux adresses : kafka:29092 dans Docker, localhost:9092 depuis l'hôte.

```
services:
  kafka:
    image: apache/kafka:4.0.0
    container_name: kafka
    ports: ["9092:9092"]
    environment:
      KAFKA_ADVERTISED_LISTENERS:
PLAINTEXT://kafka:29092,PLAINTEXT_HOST://localhost:9092
      # + rôles KRaft, quorum, listeners (voir TP)
```



POURQUOI RF = 1 ?
Un seul broker : aucune autre machine pour héberger une copie.

```
docker compose up -d
docker compose ps
docker compose logs -f kafka

docker exec kafka \
  /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server localhost:9092 \
  --list

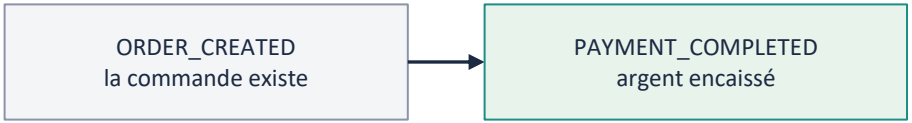
docker exec kafka \
  /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server localhost:9092 \
  --create --topic retail-orders \
  --partitions 3 \
  --replication-factor 1
```

Structure d’un événement métier

Un bon événement est identifiable, daté et sans ambiguïté sur sa nature.

```
{
  "event_id": "EVT001",
  "order_id": "ORD001",
  "customer_id": "CUS001",
  "product_id": "PROD001",
  "quantity": 2,
  "unit_price": 150.0,
  "total_amount": 300.0,
  "event_type": "PAYMENT_COMPLETED",
  "event_time": "2026-10-31T10:15:00Z"
}
```

Champ	Rôle
event_id	Identifiant unique de l’événement
order_id	Identifiant de la commande
product_id	Référence du produit
event_type	Nature de l’événement
event_time	Date réelle de production



À RETENIR
Seul PAYMENT_COMPLETED compte dans le chiffre d’affaires.

Développer un Producer Kafka avec Python

produce() met en file, flush() garantit l'envoi : l'un sans l'autre ne suffit pas.

```
from confluent_kafka import Producer
import json

producer = Producer({
    "bootstrap.servers": "localhost:9092"
})

event = {
    "event_id": "EVT001",
    "order_id": "ORD001",
    "event_type": "PAYMENT_COMPLETED",
    "total_amount": 300.0
}

producer.produce(
    topic="retail-orders",
    key=event["order_id"],
    value=json.dumps(event)
)
producer.flush()
```

bootstrap.servers
Adresse du broker : point d'entrée du cluster.

produce()
Place le message dans un tampon (asynchrone).

key
order_id : une commande reste dans une partition.

flush()
Attend l'envoi et l'acquittement.

Développer un Consumer Kafka avec Python

On valide l'offset après le traitement : au pire on retraite, jamais on ne perd.

```
from confluent_kafka import Consumer
import json

consumer = Consumer({
    "bootstrap.servers": "localhost:9092",
    "group.id": "retail-consumer",
    "auto.offset.reset": "earliest",
    "enable.auto.commit": False
})
consumer.subscribe(["retail-orders"])

while True:
    msg = consumer.poll(1.0)
    if msg is None:
        continue
    if msg.error():
        print(msg.error())
        continue
    event = json.loads(msg.value())
    print(event)              # traitement
    consumer.commit(msg)      # valider l'offset
```

1. Se connecter à Kafka
2. S'abonner à un topic
3. Récupérer les événements
4. Décoder le JSON
5. Traiter le message
6. Valider l'offset après traitement

DÉMONSTRATION
Producer et Consumer dans deux terminaux.

Introduction à Spark Structured Streaming

Chaque nouvel événement devient une nouvelle ligne d'une table non bornée.

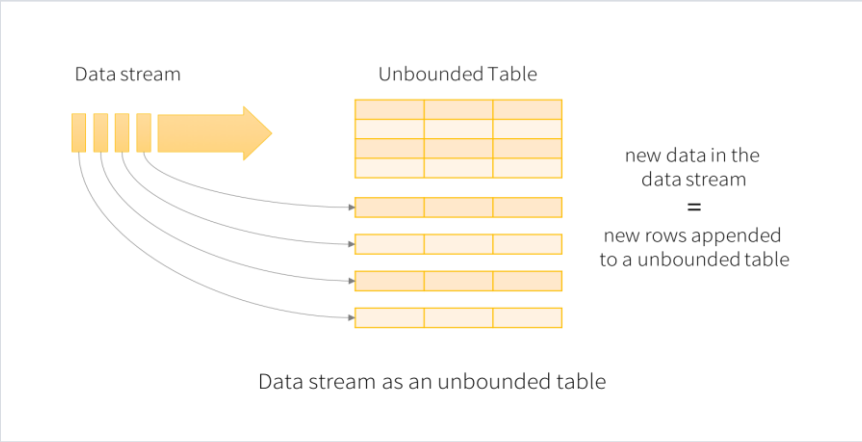


Figure — documentation Apache Spark — table non bornée

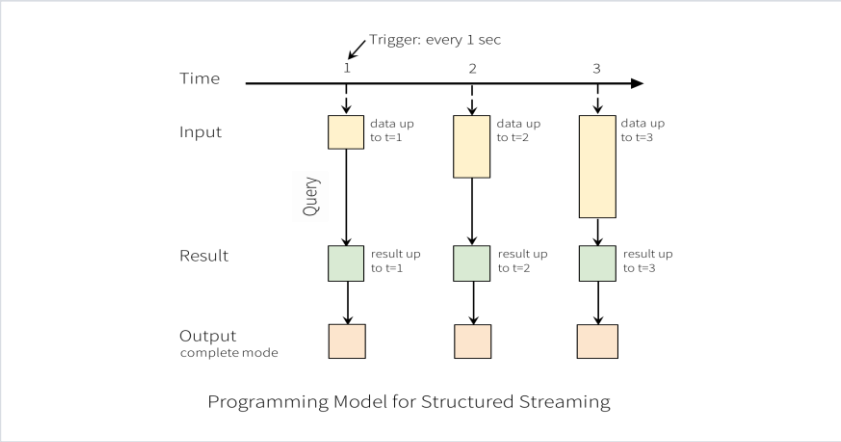


Figure — documentation Apache Spark — un trigger = un micro-batch

Output mode : Append
Seulement les nouvelles lignes finales.

Output mode : Update
Seulement les lignes modifiées.

Output mode : Complete
Toute la table résultat à chaque trigger.

Architecture d’un traitement Spark Streaming

Le Driver planifie, les Executors exécutent chaque micro-batch.

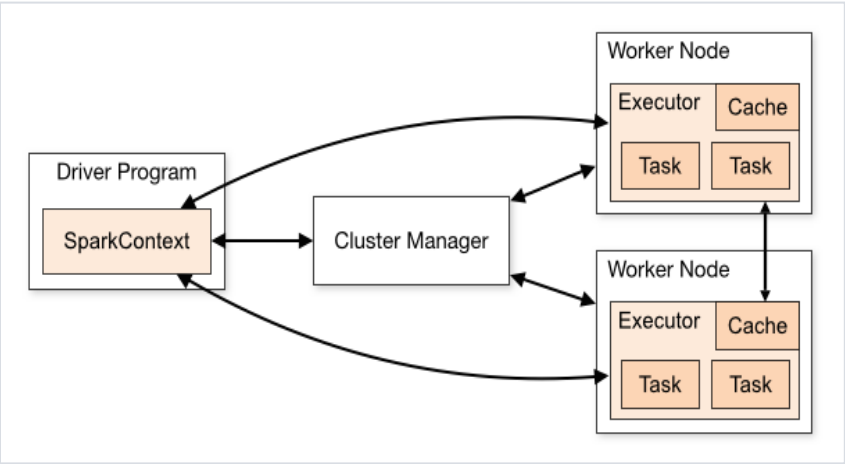
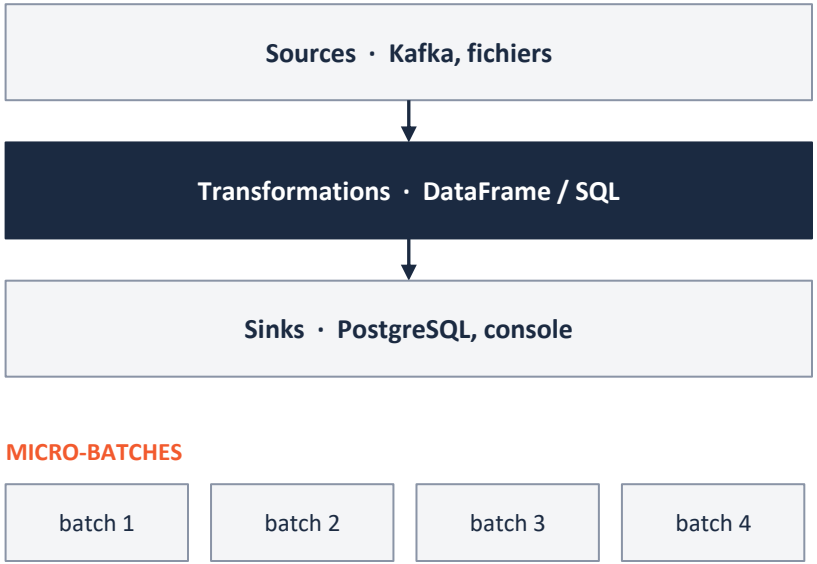


Figure — documentation Apache Spark — Driver, Executors



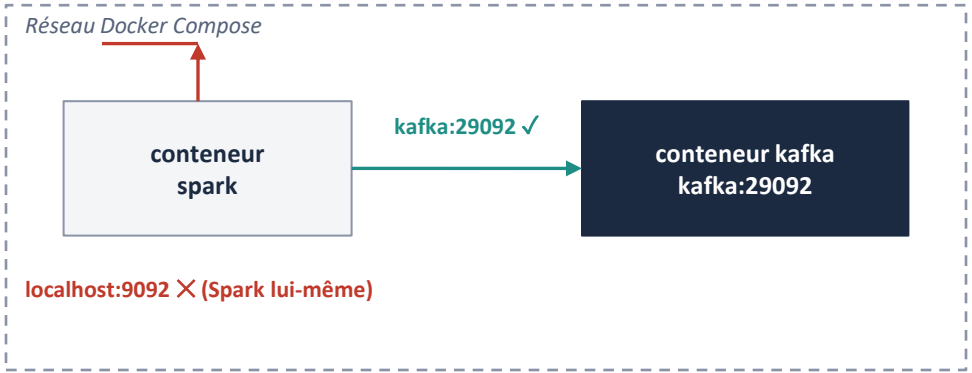
LIEN AVEC LA SÉANCE 4

Mêmes DataFrames, mêmes transformations : seule la source change.

Connexion de Spark à Kafka

Dans un conteneur, localhost désigne le conteneur lui-même : utiliser l'adresse du broker.






```
df = (
  spark.readStream
  .format("kafka")
  .option("kafka.bootstrap.servers",
          "kafka:29092")
  .option("subscribe", "retail-orders")
  .option("startingOffsets", "earliest")
  .load()
)
```

POINT TRÈS IMPORTANT
kafka:29092 dans le réseau Docker Compose, si le service et son listener sont configurés ainsi.

Parsing JSON et transformations

Kafka livre des octets : six étapes pour obtenir des colonnes typées et fiables.

1. Lire la colonne value
2. Convertir les octets en chaîne
3. Définir un schéma
4. Extraire les champs JSON
5. Filtrer les événements invalides
6. Ajouter les colonnes calculées

```
from pyspark.sql.functions import (
    col, from_json, to_date)

schema = StructType([
  StructField("event_id", StringType()),
  StructField("order_id", StringType()),
  StructField("category", StringType()),
  StructField("total_amount", DoubleType()),
  StructField("event_type", StringType()),
  StructField("event_time", TimestampType())])

events = (
  df.select(from_json(
        col("value").cast("string"),
        schema).alias("event"))
    .select("event.*")
    .filter(col("event_id").isNotNull())
    .withColumn("event_date",
                to_date("event_time")))
```

Agrégations et fenêtres temporelles

Exemple métier : chiffre d'affaires par catégorie de produits, toutes les cinq minutes.

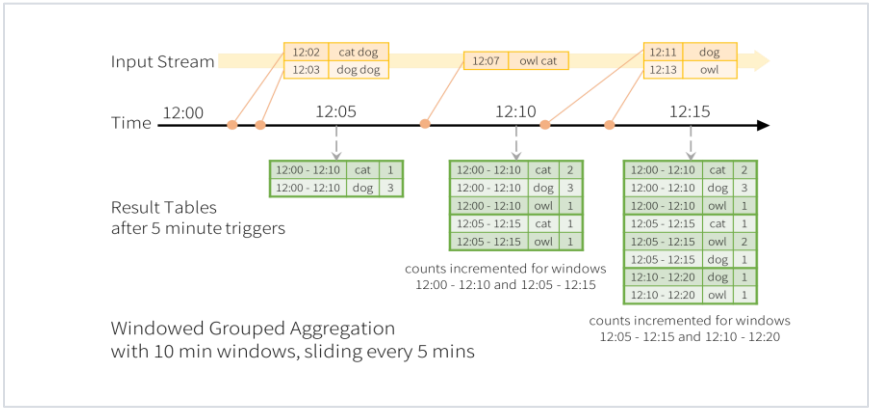
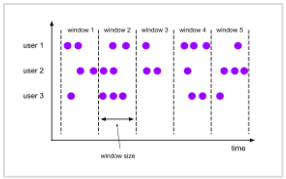
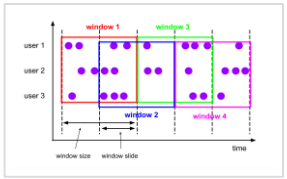


Figure — documentation Apache Spark — fenêtres glissantes

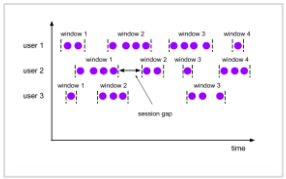
```
aggregates = (
  events
  .filter(col("event_type")
          == "PAYMENT_COMPLETED")
  .groupBy(
     window(col("event_time"), "5 minutes"),
     col("category"))
  .agg(sum("total_amount")
       .alias("revenue")))
```



Tumbling



Sliding



Session

Figures : documentation Apache Flink (concept identique en Spark)

FONCTIONS À MAÎTRISER

filter()	select()	withColumn()	groupBy()
agg()	sum()	count()	window()

Event Time, Processing Time et Watermark

Un événement en retard peut encore compter, tant que le watermark ne l'a pas dépassé.

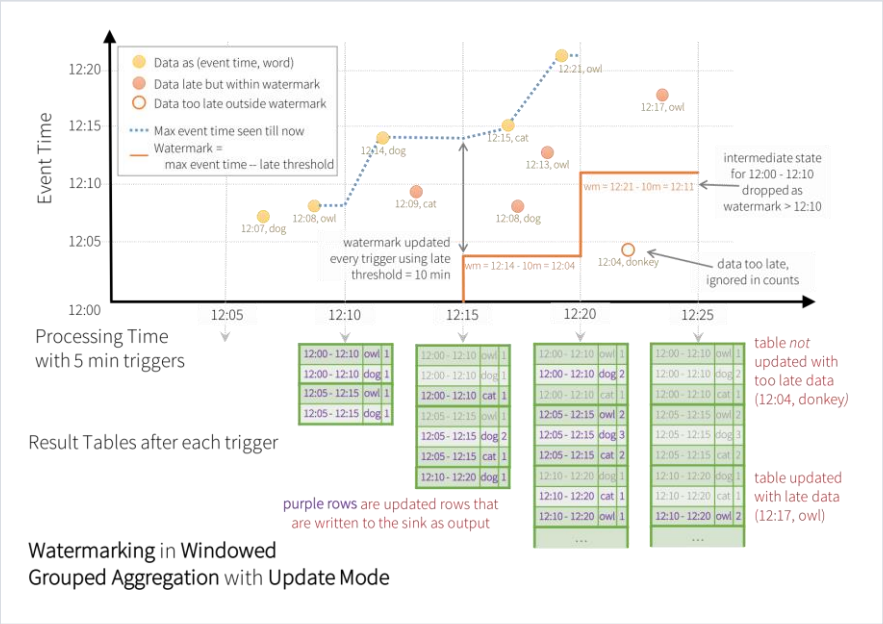


Figure — documentation Apache Spark — watermark, mode Update

```
events = events.withWatermark(
    "event_time", "10 minutes")
```

Event time
Heure réelle du paiement.

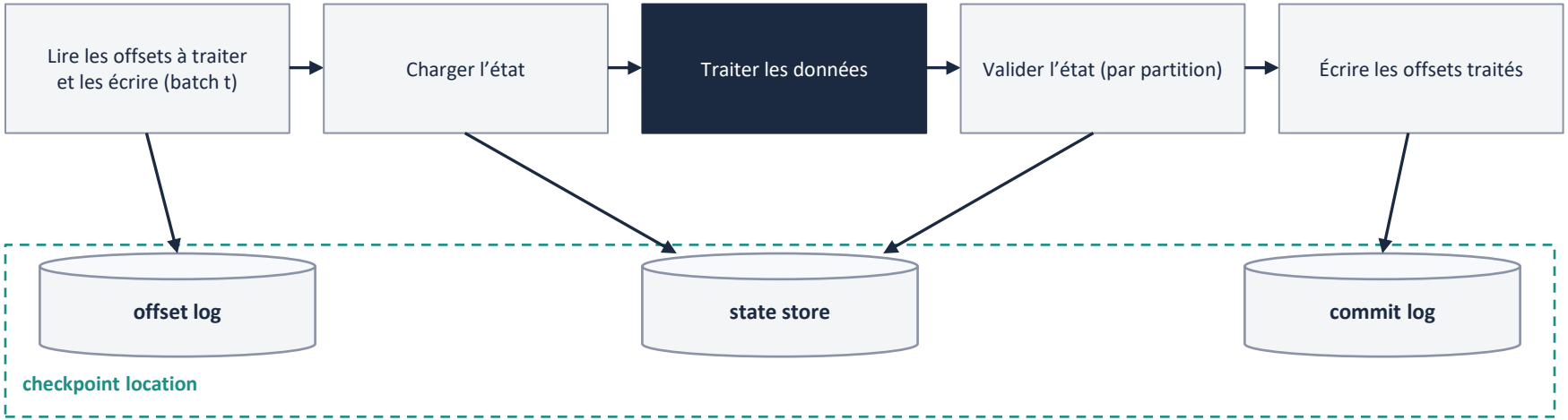
Processing time
Heure à laquelle Spark traite l'événement.

Watermark
Seuil de retard accepté ; purge l'état des agrégations.

EXEMPLE
Paiement de 10:02 reçu à 10:07 : watermark 09:57 < 10:02, il est intégré.

Checkpointing et reprise après incident

Le checkpoint conserve les offsets et l'état : Spark reprend là où il s'est arrêté.

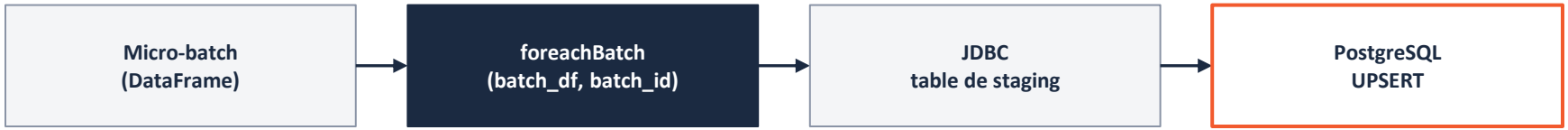



```
query = (aggregates.writeStream
  .outputMode("update")
  .option("checkpointLocation",
          "/checkpoints/retailpulse")
  .start())
```

- Le checkpoint conserve ce qu'il faut pour reprendre.
- Spark reprend après une interruption.
- Kafka conserve les événements (rétention).
- Le sink doit être conçu pour éviter les doublons.

Écrire les résultats dans PostgreSQL

foreachBatch applique une logique personnalisée à chaque micro-batch ; l'UPSERT évite les doublons.




```python
def write_to_postgres(batch_df, batch_id):
    # 1. transformation du batch
    # 2. écriture dans la table de staging
    (batch_df.write.format("jdbc")
       .option("url", JDBC_URL)
       .option("dbtable", "staging_revenue")
       .mode("overwrite").save())
    # 3. UPSERT vers la table finale
```

```sql
INSERT INTO revenue_by_window
  (window_start, category, revenue)
SELECT window_start, category, revenue
FROM staging_revenue
ON CONFLICT (window_start, category)
DO UPDATE
  SET revenue = EXCLUDED.revenue;
```

ATTENTION

Un INSERT en append duplique après reprise : UPSERT ou clé d'idempotence.

Performances et supervision du pipeline

On ne règle que ce que l'on mesure : débit, lag, durée de batch, ressources.

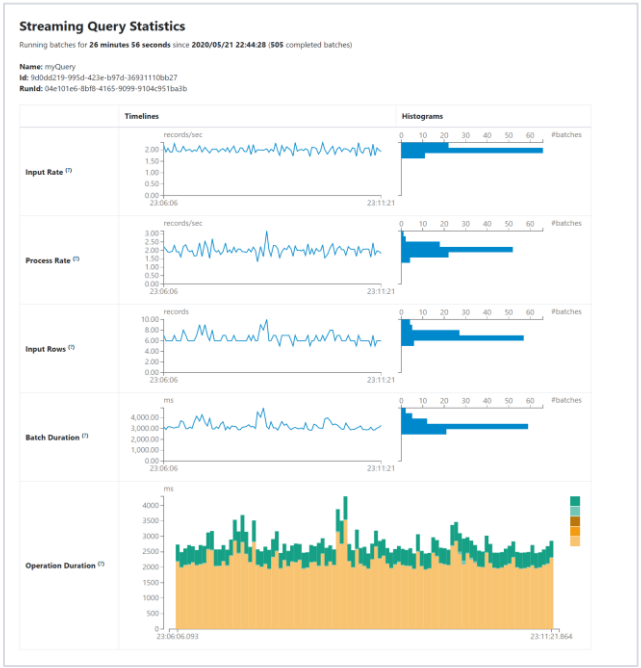


Figure — Spark UI, Structured Streaming

Indicateur	Signification
Throughput	Événements traités par seconde
Consumer lag	Retard de consommation Kafka
Processing time	Durée d'un micro-batch
Input rows	Lignes reçues
Processed rows	Lignes traitées
CPU / RAM	Ressources consommées

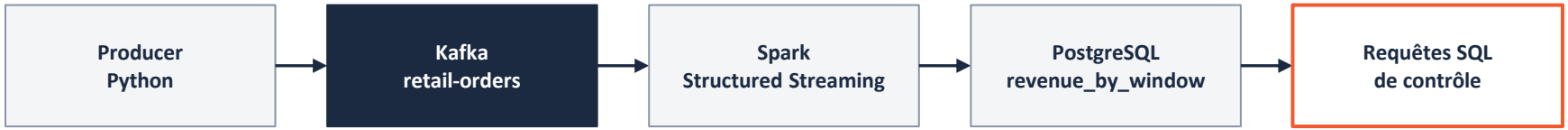
OUTILS

Kafka consumer groups	Spark UI	docker stats	Logs des conteneurs

```
kafka-consumer-groups.sh --bootstrap-server kafka:29092 --describe --group retail-consumer
```

Data ingestion : pipeline retail-orders

Du producteur Python à PostgreSQL : un pipeline complet, mesuré et résistant aux pannes.



1. Démarrer Kafka, Spark et PostgreSQL (Docker Compose)
2. Créer le topic retail-orders (3 partitions)
3. Produire des événements PAYMENT_COMPLETED
4. Lire le topic avec Spark (kafka:29092)
5. Parser le JSON et agréger par fenêtre de 5 minutes
6. Écrire dans PostgreSQL avec UPSERT
7. Arrêter puis relancer Spark avec le checkpoint
8. Mesurer débit, lag et durée de batch

À VÉRIFIER
Aucun doublon dans PostgreSQL après redémarrage de Spark.